Timers and PWM

Prof. Gerardo Flores

April 2026

Microprocessor Systems (CSCE 3340 - SENG 3345)

TAMIU

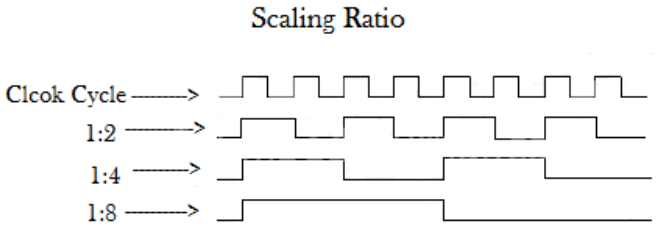
Timers

1. A timer can be used as a precise time base.
2. First, the **prescaler** defines the tick duration.
3. Then, the auto-reload register defines the **period**.
4. When the timer reaches that period, an update **event** occurs.
5. In this example, that event is used to toggle an LED

Important Registers for This Example

- **PSC** → sets timer speed
- **ARR** → sets period
- **CNT** → current timer count
- **SR** → contains timer status flags
- **UIF** → indicates the timer period has completed
- **CR1** → starts the timer

Timers



•The timer divides the clock this way:

$$f_{CNT} = \frac{f_{INT}}{PSC + 1}$$

timer TIM3 = 48MHz

the frequency at which the counter CNT increments [how often a timer tick occurs]

`TIM3->PSC`

$$f_{update} = \frac{f_{INT}}{(PSC + 1)(ARR + 1)}$$

number of ticks before overflow

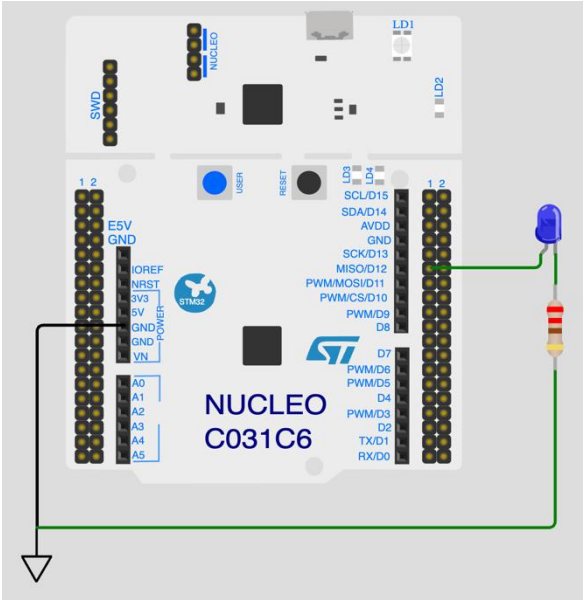
$$T_{timer} = \frac{(PSC + 1)(ARR + 1)}{f_{INT}}$$

Timer-Based LED Blinking (STM32 – Algorithm)

Objective

Use **TIM3** as a precise time base to generate a periodic event and **toggle an LED every 500 ms**.

Step-by-Step Timer Configuration Algorithm



Step 1 — Enable GPIO clock

Enable the clock for the port that contains the LED pin.

```
RCC->IOPENR |= RCC_IOPENR_GPIOAEN;
```

This allows configuration of **PA5**.

Step 2 — Configure LED pin as output

Set PA5 to general-purpose output mode.

```
GPIOA->MODER &= ~(3U << (5U * 2U));
GPIOA->MODER |=  (1U << (5U * 2U));
```

Now PA5 can directly drive the LED.

Step 3 — Enable timer clock

Turn on the timer peripheral.

```
RCC->APBENR1 |= RCC_APBENR1_TIM3EN;
```

Without this step, TIM3 cannot operate.

Step 4 — Configure the prescaler (PSC)

Reduce the timer clock from **48 MHz** to **1 kHz**.

```
TIM3->PSC = 48000 - 1;
```

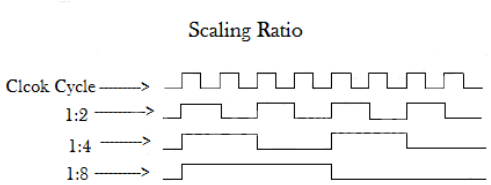
This gives:

$$48\,\text{MHz}/48000 = 1000\,\text{Hz}$$

So the timer tick becomes:

$$1\text{ tick} = 1\text{ ms}$$

$$f_{CNT} = \frac{f_{INT}}{PSC + 1}$$



Step 5 — Configure the auto-reload register (ARR)

$$f_{update} = \frac{f_{INT}}{(PSC + 1)(ARR + 1)}$$

number of ticks before overflow

Set the timer period to **500 ms**.

```
TIM3->ARR = 500 - 1;
```

Since each tick is 1 ms:

$$500 \times 1 \text{ ms} = 500 \text{ ms}$$

This means the timer completes one full cycle every 500 ms.

$$T_{timer} = \frac{1}{f_{update}} = \frac{(48000)(500)}{48 \times 10^6} = 0.5s$$

Step 6 — Reset the counter

Initialize the timer count value.

```
TIM3->CNT = 0;
```

This ensures the timer starts counting from zero.

Step 7 — Apply the timer configuration

Force the timer to load the configured register values.

```
TIM3->EGR |= TIM_EGR_UG;
```

This updates the timer immediately.

Step 8 — Start the timer

Enable the timer counter.

```
TIM3->CR1 |= TIM_CR1_CEN;
```

Now TIM3 begins counting from 0 to ARR repeatedly.

At this point, the configuration is finished.

We start using the Timer here:

Step 9 — Wait for the update event

Monitor the timer until the period has elapsed.

```
while (!(TIM3->SR & TIM_SR_UIF));
```

The **UIF flag** is set when the timer reaches ARR.

Step 10 — Clear the update flag

Reset the flag so the next period can be detected.

```
TIM3->SR &= ~TIM_SR_UIF;
```

This prepares the timer for the next cycle.

Step 11 — Toggle the LED

Change the LED state every time the timer period finishes.

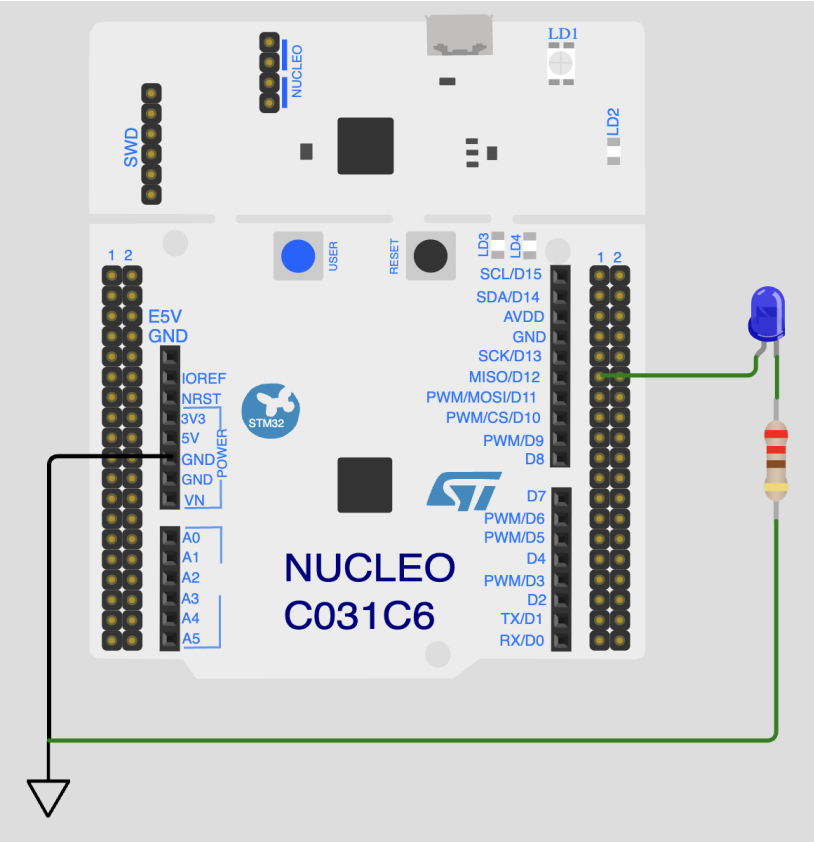
```
GPIOA->ODR ^= (1U << 5);
```

This makes the LED blink periodically.

Solution

timer_bare_metal

```c
#include "stm32c031xx.h"

static void gpio_led_init(void)
{
    /* Enable GPIOA clock */
    RCC->IOPENR |= RCC_IOPENR_GPIOAEN;

    /* PA5 -> output mode */
    GPIOA->MODER &= ~(3U << (5U * 2U));
    GPIOA->MODER |=  (1U << (5U * 2U));
}

static void tim3_init_500ms(void)
{
    /* Enable TIM3 clock */
    RCC->APBENR1 |= RCC_APBENR1_TIM3EN; //Advanced Peripheral Bus -> APB

    /*
        Timer clock = 48 MHz

        Step 1: Prescaler
        48 MHz / 48000 = 1000 Hz
        -> 1 tick = 1 ms
    */
    TIM3->PSC = 48000 - 1;

    /*
        Step 2: Auto-reload register

        500 ticks × 1 ms = 500 ms
    */
    TIM3->ARR = 500 - 1;

    /* Reset counter */
    TIM3->CNT = 0;
```



•This is just part of the code, go to GitHub and download the project

What the Timer Is Doing Internally

The timer repeatedly performs this sequence:

$$\mathrm{CNT}: 0 \rightarrow 1 \rightarrow 2 \rightarrow \cdots \rightarrow \mathrm{ARR}$$

When:

$$\mathrm{CNT} = \mathrm{ARR}$$

the timer:

- sets the **UIF flag**
- resets the counter
- starts counting again

This creates a precise periodic event.

Key Timing Result

With this configuration:

- **Timer tick** = 1 ms
- **ARR** = 500
- **Timer period** = 500 ms

So the LED changes state every:

$$500\ ms$$

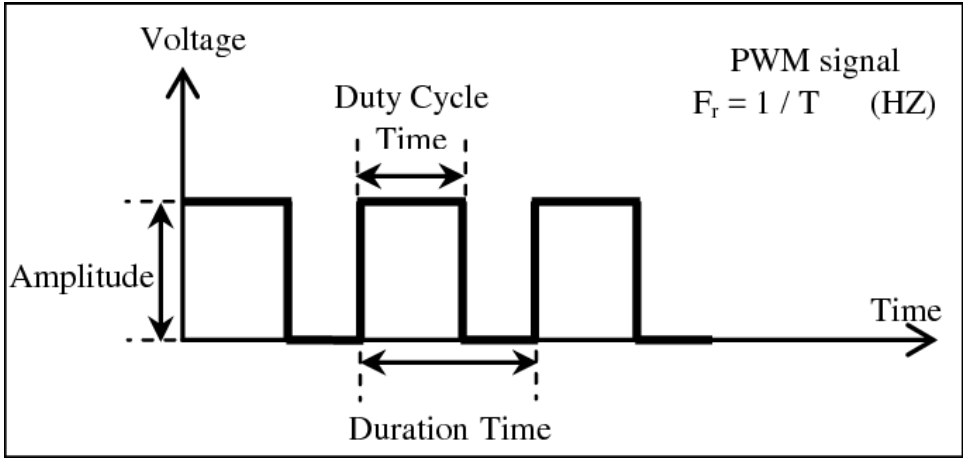
If the LED toggles every 500 ms, one full ON/OFF cycle takes 1 second.

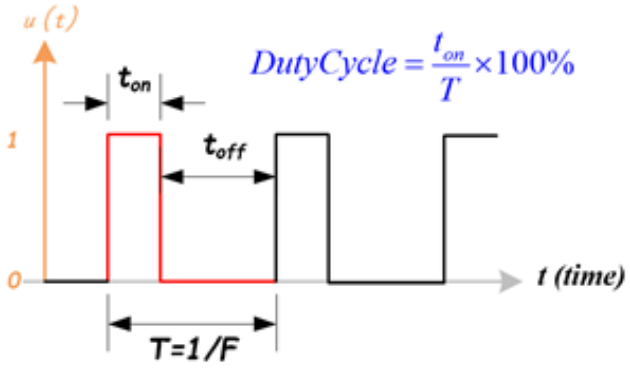
Pulse-Width Modulation

It means modulating/varying the width of the pulse only, not the frequency.

The speed of the motor depends on three factors: **(a)** load, **(b)** voltage, and **(c)** current.

By varying (modulating) the width of the pulse applied to the DC motor we can increase or decrease the amount of power provided to the motor, thereby increasing or decreasing the motor speed.





PWM Signal Generation Workflow

Configuring Timer for Servo Control

Prescaler (PSC)

PSC = 480 − 1

Timer Clock:

48 MHz / 480 = 100 kHz

Tick = 10 µs

Auto-Reload Register (ARR)

ARR = 2000 − 1

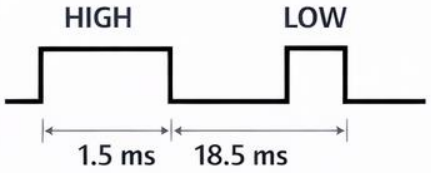
Period: **20 ms**

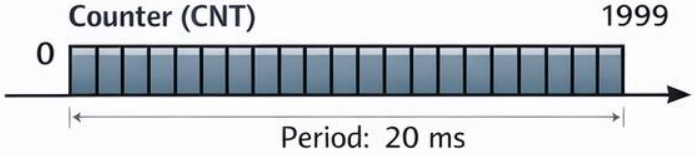
Frequency: 50 Hz

Compare Register (CCR1)

CCR1 = 150

Pulse Width: 150 × 10 µs = 1.5 ms





Clock Divider → **Set Period** → **Set Pulse Width**

100 kHz Timer → 20 ms Period → 1.5 ms Pulse

PWM Generation for Servo Control (STM32)

Objective

Generate a **50 Hz PWM signal** on **PA6 (TIM3_CH1)** where the **pulse width (1–2 ms)** controls the servo position.

Step-by-Step PWM Configuration Algorithm

Step 1 — Enable GPIO clock

Enable the clock for the port that contains the PWM pin.

```
RCC->IOPENR |= RCC_IOPENR_GPIOAEN;
```

This allows configuration of pin **PA6**.

Step 2 — Configure pin as Alternate Function

Set PA6 to Alternate Function mode.

```
GPIOA->MODER &= ~(3U << (6U * 2U));
GPIOA->MODER |=  (2U << (6U * 2U));
```

Alternate Function mode lets peripherals control the pin.

GPIOA Alternate Function Register LOW (AFRL)

Step 3 — Select timer function for the pin

Connect PA6 to **TIM3 Channel 1**.

```
GPIOA->AFR[0] &= ~(0xFU << (6U * 4U));
GPIOA->AFR[0] |=  (1U   << (6U * 4U));
```

Now PA6 outputs the PWM signal generated by TIM3_CH1.

We need an output signal instead of an internal one

AF1 → TIM3_CH1 output

Step 4 — Enable timer clock

Turn on the timer peripheral.

```
RCC->APBENR1 |= RCC_APBENR1_TIM3EN;
```

Without this step, the timer cannot operate.

Step 5 — Configure timer frequency (Prescaler)

Reduce timer clock from 48 MHz to 100 kHz.

```
TIM3->PSC = 480 - 1;
```

Timer tick becomes:

$$1 \text{ tick} = 10\ \mu s$$

Table 13. Port A alternate function mapping (AF0 to AF7)

Port	AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7
PA0	-	USART2_CTS	TIM16_CH1	-	USART1_TX	TIM1_CH1	-	-
PA1	SPI1_SCK/I2S1_CK	USART2_RTS_DE_CK	TIM17_CH1	-	USART1_RX	TIM1_CH2	I2C1_SMBA	EVENTOUT
PA2	SPI1_MOSI/I2S1_SD	USART2_TX	TIM16_CH1N	TIM3_ETR	-	TIM1_CH3	-	-
PA3	-	USART2_RX	TIM1_CH1N	-	-	TIM1_CH4	-	EVENTOUT
PA4	SPI1_NSS/I2S1_WS	USART2_TX	TIM1_CH2N	-	TIM14_CH1	TIM17_CH1N	-	EVENTOUT
PA5	SPI1_SCK/I2S1_CK	USART2_RX	TIM1_CH3N	-	-	TIM1_CH1	-	EVENTOUT
PA6	SPI1_MISO/I2S1_MCK	TIM3_CH1	TIM1_BKIN	-	-	TIM16_CH1	-	-
PA7	SPI1_MOSI/I2S1_SD	TIM3_CH2	TIM1_CH1N	-	TIM14_CH1	TIM17_CH1	-	-
PA8	MCO	USART2_TX	TIM1_CH1	-	-	-	-	EVENTOUT
PA9	MCO	USART1_TX	TIM1_CH2	TIM3_ETR	-	-	I2C1_SCL	EVENTOUT
PA10	-	USART1_RX	TIM1_CH3	MCO2	-	TIM17_BKIN	I2C1_SDA	EVENTOUT
PA11	SPI1_MISO/I2S1_MCK	USART1_CTS	TIM1_CH4	-	-	TIM1_BKIN2	-	-
PA12	SPI1_MOSI/I2S1_SD	USART1_RTS_DE_CK	TIM1_ETR	-	-	I2S_CKIN	-	-
PA13	SWDIO	IR_OUT	-	TIM3_ETR	USART2_RX	-	-	EVENTOUT
PA14	SWCLK	USART2_TX	-	-	-	-	-	EVENTOUT
PA15	SPI1_NSS/I2S1_WS	USART2_RX	TIM1_CH1	MCO2	USART1_RTS_DE_CK	-	-	EVENTOUT

Step 6 — Configure PWM period

Set PWM period to **20 ms (50 Hz)**.

```
TIM3->ARR = 2000 - 1;
```

Because:

$$2000 \times 10\ \mu s = 20\ ms$$

This matches the standard servo control frequency.

Step 7 — Select PWM mode

Enable PWM Mode 1 on Channel 1.

```
TIM3->CCMR1 |= (6U << 4);
```

PWM Mode 1 behavior:

Output HIGH while

$$CNT < CCR1$$

Output LOW otherwise.

Step 8 — Enable preload registers

Activate buffered updates for stable PWM operation.

```
TIM3->CCMR1 |= TIM_CCMR1_OC1PE;
TIM3->CR1   |= TIM_CR1_ARPE;
```

New values take effect at the next update event.

Use preload registers when PWM duty cycle changes during execution.

Otherwise they are optional.

Step 9 — Enable channel output

Allow timer output to reach the pin.

```
TIM3->CCER |= TIM_CCER_CC1E;
```

→ Enables TIM3 Channel 1 output on the pin.

Without this step, no signal appears at PA6.

Step 10 — Set initial pulse width

Initialize servo to center position.

```
TIM3->CCR1 = 150;
```

Since:

$$150 \times 10\ \mu s = 1.5\ ms$$

→ Remember that 1 tick = 10 μs

This corresponds to the neutral servo position.

CCR1 sets the PWM pulse width.

```
100 × 10µs = 1 ms
200 × 10µs = 2 ms
```

CCR1 = 150 → 1.5 ms pulse → servo center position

Step 11 — Apply configuration

Force update of timer registers.

```
TIM3->EGR |= TIM_EGR_UG;
```

Ensures PSC, ARR, and CCR values are loaded immediately.

Step 12 — Start timer

Begin PWM signal generation.

```
TIM3->CR1 |= TIM_CR1_CEN;
```

Timer now continuously generates PWM on PA6.

How PWM Controls the Servo Position

Pulse width is determined by:

$$\text{Pulse Width} = CCR1 \times 10\ \mu s$$

Typical values:

CCR1	Pulse Width	Servo Position
100	1.0 ms	Minimum angle
150	1.5 ms	Center
200	2.0 ms	Maximum angle

PWM period:

$$T = \frac{(PSC + 1)(ARR + 1)}{f_{timer}}$$

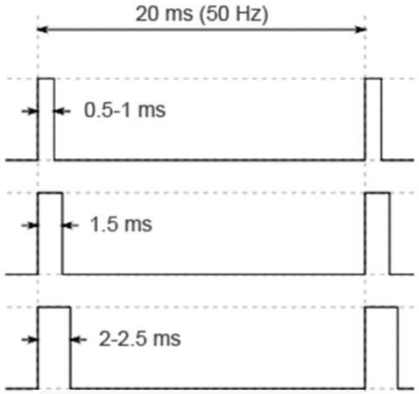
Pulse width:

$$t_{HIGH} = CCR1 \times 10\ \mu s$$

Servo frequency:

$$f = 50\ Hz$$

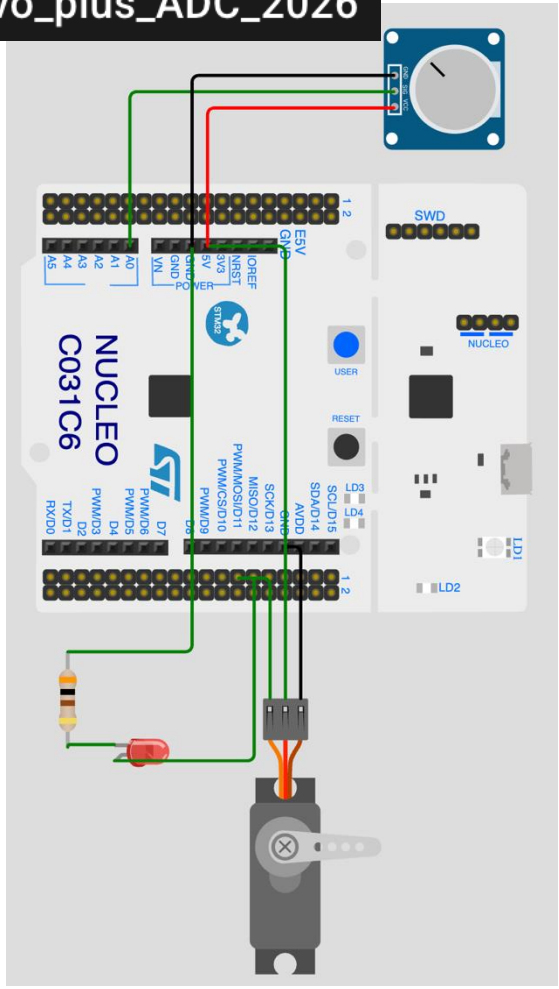
Solution



100 × 10µs = 1 ms
200 × 10µs = 2 ms

```c
int main(void)
{
    uint16_t adc_value;
    uint32_t ccr_value;

    pwm_servo_init();
    adc_init_pa0();

    while (1)
    {
        adc_value = adc_read();

        /*
          Map ADC 0..4095 to servo pulse 1.0..2.0 ms
          CCR1 100..200
        */
        ccr_value = SERVO_MIN +
                    ((uint32_t)adc_value * (SERVO_MAX - SERVO_MIN)) / 4095U;

        TIM3->CCR1 = ccr_value;
    }
}
```

Servo_plus_ADC_2026



•This is just part of the code, go to GitHub and download the project